UNIVERSIDAD SERGIO ARBOLEDA

Escuela de Ciencias Exactas e Ingeniería

Ingeniería en Ciencias de la Computación e Inteligencia Artificial (ICCIA)

Informe Técnico Final de Ingeniería

Entrega Final y Sustentación (Sesión 5)

Paradigmas de Programación (SIST0176) - Semestre III

Grupo 4: Segmentación y Procesamiento de Imágenes Médicas por Visión Artificial

Integrantes: [Nombre 1], [Nombre 2], [Nombre 3], [Nombre 4] (completar)

Docente: [completar] | Fecha: noviembre de 2026

1. Resumen

Se desarrolló un sistema multiparadigma para procesar tomografías: un kernel imperativo en **C++17 con OpenMP** calcula bordes por convolución 2D; un dominio **orientado a objetos en Java 17** (Factory Method, DIP, inmutabilidad) orquesta el análisis; y un pipeline **funcional en Python** normaliza intensidades y segmenta regiones con recursión de cola y contenedores Option. El sistema detecta correctamente los tumores sintéticos de los tres estudios de prueba y pasa 30 pruebas automáticas (12 Python, 18 Java). La versión paralela produce resultados idénticos a la secuencial.

Limitaciones importantes: (1) los datos son TAC sintéticas, no clínicas; (2) los benchmarks incluidos se midieron en un entorno con **1 solo hilo de hardware**, por lo que no pueden mostrar aceleración: la sección 7 explica cómo obtener la tabla de speedup real en el equipo del grupo.

2. Caso de estudio y requerimientos

Requerimiento de la especificación (Grupo 4)	Implementación	Dónde
Kernel de convolución O(W·H·K²), localidad de caché por filas contiguas, OpenMP	convolucionar(): padding contiguo, bucle por filas, #pragma omp parallel for	cpp/convolucion.cpp
Factory Method para instanciar analizadores médicos	FabricaAnalizador (Creator) y fábricas Tumoral / Óseo	java/src/tac/analisis
Copias defensivas en EstudioMedico	clone() al entrar y al salir; getPixeles() devuelve copia	dominio/EstudioMedico.java

Requerimiento de la especificación (Grupo 4)	Implementación	Dónde
Inversión de dependencias con RepositorioImagen	Interfaz RepositorioImagen; ServicioDiagnostico depende solo de ella	repositorio/, analisis/ServicioDiagnostico.java
Mapeo puro de intensidad de píxeles	ventana() currificada + map	funcional/pixeles.py
Recursión de cola (TCO) para segmentación	_crecer / _etiquetar con trampolín	funcional/segmentacion.py
Contenedores Option / Maybe	Some / Nothing; parsear_pgm y region_mayor	funcional/option.py
Functional Core, Imperative Shell; benchmarks; thread safety	cli.py como única capa con E/S; make bench; pruebas concurrentes	funcional/cli.py, cpp/bench.cpp, PruebasTac

3. Arquitectura de dominio

El diseño completo (diagramas UML con visibilidades, firmas y relaciones) está en el documento de la entrega parcial. Resumen de decisiones:

- **Dominio inmutable:** EstudioMedico es una clase final con copias defensivas; ResultadoAnalisis, RegionTumoral y ParametrosSegmentacion son *records* con listas copiadas (List.copyOf) y arreglos clonados.
- **Template Method:** AnalizadorBase.analizar() es final y fija normalizar -> bordes -> segmentar; las subclases solo entregan parámetros (OCP, LSP).
- **Factory Method:** el servicio crea analizadores a través de FabricaAnalizador sin conocer clases concretas.
- **DIP:** ServicioDiagnostico, AnalizadorBase y las fábricas dependen de interfaces; las pruebas sustituyen C++ y Python por falsos.

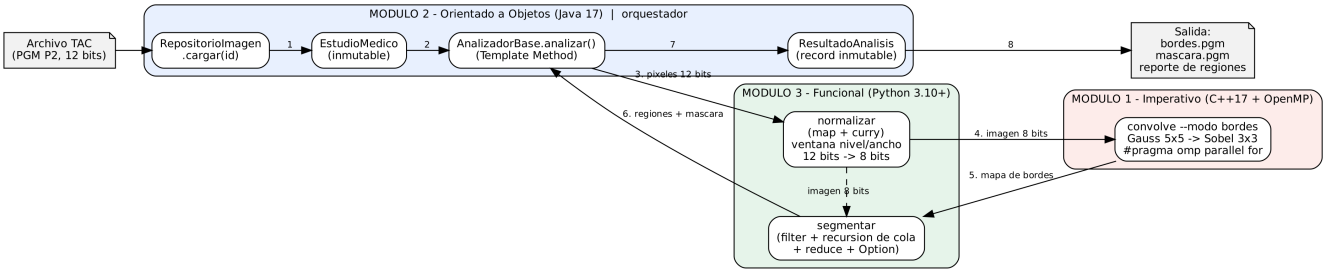


Figura 1. Flujo de datos entre los tres módulos.

4. Implementación por paradigma

4.1 Imperativo - C++17 y OpenMP

- **Memoria:** imagen y buffers en std::vector<float> (bloque contiguo en el Heap, sin fugas por RAII); el filtro gaussiano es un arreglo pequeño en el Stack.
- **Optimización:** el padding con bordes replicados quita las ramas del bucle interno; el recorrido es por filas (row-major).
- **Paralelismo:** `#pragma omp parallel for schedule(static)` sobre las filas; cada hilo escribe filas distintas, sin locks y sin carreras.
- **Interfaz:** programa convolve (PGM por stdin/stdout) y programa bench.

4.2 Orientado a objetos - Java 17

- **Inmutabilidad y aliasing:** 7 pruebas comprueban que modificar el arreglo original, el devuelto por getPixeles() o la lista de resultados no altera el estado interno.
- **Thread safety:** el servicio no tiene estado mutable; cada llamada externa usa archivos temporales propios; RepositorioImagenEnMemoria usa ConcurrentHashMap. Prueba de estrés: 8 hilos × 500 escrituras sin pérdidas; lote con 4 hilos = lote con 1 hilo.
- **Errores:** jerarquía de excepciones verificadas (TacException); un fallo en un hilo del lote se propaga al llamador.

4.3 Funcional - Python 3.10+

- **Pureza y transparencia referencial:** normalizar, mascara y segmentar no modifican sus argumentos ni hacen E/S; mismas entradas, mismas salidas (probado).
- **map / filter / reduce:** map para ventana y máscara, filter para filtrar regiones pequeñas y píxeles pendientes, reduce para estadísticas de región y región mayor.
- **Currificación:** `ventana(nivel)(ancho)(pixel)` y `en_banda(bajo)(alto)`; compose para composición de funciones.
- **Evaluación perezosa:** generador vecinos4 y filter sobre componentes se evalúan bajo demanda.
- **Recursión de cola:** Python no hace TCO; se emula con un *trampolín* (cada paso devuelve la siguiente llamada y un bucle la ejecuta), con pila constante.
- **Option:** parsear_pgm devuelve Some/Nothing en lugar de excepciones o None; region_mayor devuelve Nothing si no hay regiones (estudio sano).

5. Algoritmo de segmentación

1) Ventana nivel/ancho lleva 12 bits a 8 bits. 2) C++ calcula la magnitud del gradiente (suavizado 5×5 + Sobel). 3) Máscara = píxeles dentro de la banda de intensidad del tejido buscado **y** con borde débil (excluye las transiciones cerebro/cráneo, que caerían en la banda). 4) Componentes conexas (4 vecinos) por recursión de cola; 5) se descartan regiones menores que el área mínima; 6) se calculan área, caja y centroide.

Estudio	Verdad (fantoma)	Área real	Área detectada	Detectado/real	Centroide detectado
tac_01	2 tumores (r=13 en (100,108); r=8 en (165,150))	529 px y 197 px	421 px y 140 px	80% y 71%	(99.6, 107.9) y (165.0, 150.0)
tac_02	1 tumor irregular (dos círculos unidos)	646 px	536 px	83%	(124.1, 163.5)
tac_03	Sano (sin tumor)	0	Sin regiones (Option = Nothing)	-	-

Las áreas detectadas son menores (70-80 %) porque se excluyen los píxeles del borde del tumor para separar regiones; los centroides coinciden con los reales (error menor que 0.5 px en tac_01). Los umbrales (banda 100-170, borde < 30, área mínima 30) se calibraron manualmente sobre estos fantomas y **no se validaron con imágenes reales**.

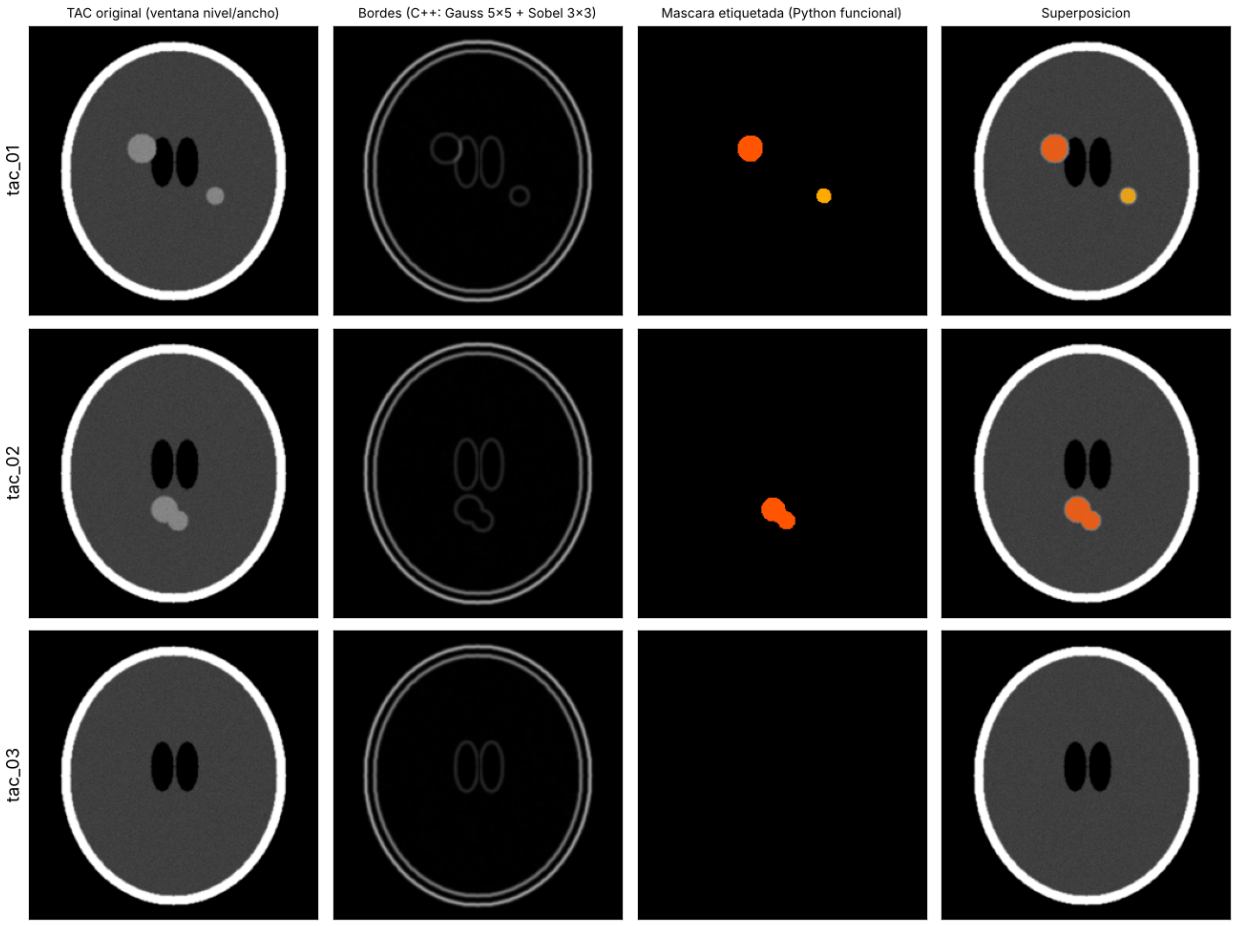


Figura 2. Resultados: original, bordes (C++), máscara etiquetada (Python) y superposición.

6. Análisis Big O

Demostraciones formales en el informe de avance (entrega parcial). Resumen:

Componente	Tiempo	Espacio
Convolución K×K (C++)	$\Theta(W \cdot H \cdot K^2)$	$\Theta(W \cdot H)$
Detección de bordes (K fijos 5 y 3)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$
Versión paralela con p hilos	$\Theta(W \cdot H \cdot K^2/p + W \cdot H)$	$\Theta(W \cdot H)$
Normalizar y máscara (Python)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$
Componentes conexas con estructuras inmutables	$O(M^2)$ peor caso	$O(M)$
Copias defensivas y E/S (Java)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$

7. Benchmarks de paralelismo (Secuencial vs Paralelo)

Método: kernel de caja K×K sobre imagen aleatoria; se toma el mejor de 3 ejecuciones; se verifica que el resultado paralelo sea idéntico al secuencial. **Entorno de esta medición: hilos máximos de OpenMP = 1**.

Tamaño	K	Hilos	T seq (ms)	T par (ms)	Speedup	Eficiencia	Resultado igual
512x512	3	2	1.5	1.4	1.04x	52%	si

Tamaño	K	Hilos	T seq (ms)	T par (ms)	Speedup	Eficiencia	Resultado igual
512x512	3	4	1.5	1.6	0.92x	23%	si
512x512	5	2	4.1	3.8	1.10x	55%	si
512x512	5	4	4.1	3.7	1.11x	28%	si
512x512	9	2	10.0	10.4	0.96x	48%	si
512x512	9	4	10.0	10.0	1.00x	25%	si
1024x1024	3	2	6.1	5.6	1.09x	54%	si
1024x1024	3	4	6.1	5.5	1.11x	28%	si
1024x1024	5	2	14.7	15.3	0.96x	48%	si
1024x1024	5	4	14.7	15.0	0.98x	25%	si
1024x1024	9	2	41.7	43.5	0.96x	48%	si
1024x1024	9	4	41.7	44.4	0.94x	23%	si
2048x2048	3	2	30.2	23.6	1.28x	64%	si
2048x2048	3	4	30.2	23.8	1.27x	32%	si
2048x2048	5	2	61.9	62.6	0.99x	49%	si
2048x2048	5	4	61.9	61.5	1.01x	25%	si
2048x2048	9	2	160.9	161.2	1.00x	50%	si
2048x2048	9	4	160.9	163.7	0.98x	25%	si

Interpretación honesta: con 1 hilo de hardware, pedir 2 o 4 hilos no puede acelerar (speedup ≈ 1.0x; las pequeñas diferencias son ruido de medición). Estas cifras **no demuestran** aceleración; demuestran que la versión paralela es correcta e idéntica a la secuencial. Para el speedup real, ejecutar `make bench` en el computador del grupo (con 4 o más núcleos), y volver a correr `python3 docs/herramientas/generar_documentos.py`, que incorpora la nueva tabla a este informe.

Cota teórica esperada (ley de Amdahl, sección 3.4 del informe de avance; modelo, no medición):

Kernel	f (secuencial)	p = 2	p = 4	p = 8
K = 3	10.0%	1.82x	3.08x	4.71x
K = 5	3.8%	1.93x	3.59x	6.30x
K = 9	1.2%	1.98x	3.86x	7.37x

8. Control de efectos secundarios

Efecto secundario	Dónde ocurre	Cómo se aísla
Lectura/escritura de archivos y procesos	Java: RepositorioImagenArchivo, FormatoPGM, ProcesoExterno	Detrás de interfaces; el dominio no hace E/S.
stdin/stdout, argumentos, códigos de salida	Python: solo cli.py (Imperative Shell)	Los módulos pgm, pixeles, segmentacion y option son puros (Functional Core).
Escritura de arreglos en paralelo	C++: convolucionar()	Cada hilo escribe filas disjuntas; la entrada solo se lee.
Estado compartido entre hilos	Java: RepositorioImagenEnMemoria	ConcurrentHashMap con valores inmutables.
Archivos temporales	ProcesoExterno	Nombres únicos por llamada; se borran en finally.

9. Pruebas y validación

Suite	Pruebas	Qué cubre
Python (unittest)	12	Option, pureza, currificación, compose, máscara, componentes, pila con 20 000 pasos, PGM
Java (PruebasTac)	18	Aliasing, inmodificabilidad, Factory Method, integración real de 3 módulos, DIP con falsos, lote concurrente, estrés, propagación de errores
C++ (bench)	-	Igualdad bit a bit secuencial vs paralelo en cada configuración

10. Limitaciones y trabajo futuro

- Datos sintéticos: faltaría validar con imágenes reales (DICOM) y umbrales adaptativos.
- Integración por procesos y archivos: simple y portable, pero con sobrecosto; JNI/JNA o sockets lo reducirían.
- Segmentación inmutable $O(M^2)$: estructuras persistentes la bajarían a $O(M \log M)$.
- Gauss 5×5 es separable: dos pasadas 1D bajarían de 25 a 10 multiplicaciones por píxel; también se puede paralelizar el padding (hoy secuencial).
- Medir speedup en un equipo multinúcleo (pendiente, ver sección 7).

11. Conclusiones

Cada paradigma resolvió la parte donde más aporta: el imperativo, el control de memoria y el paralelismo; el orientado a objetos, el modelado del dominio con invariantes protegidos y bajo acoplamiento; el funcional, la transformación de datos sin efectos secundarios. La integración por contratos permitió probar cada módulo por separado y luego juntos.

Apéndice. Cómo ejecutar

```
make all      # compila C++ y Java
make test     # 12 pruebas Python + 18 pruebas Java
make demo     # analiza tac_01 y un lote concurrente
make bench    # benchmark secuencial vs paralelo (docs/benchmarks.md)
python3 datos/generar_tac.py   # regenera las TAC sinteticas
```